

프로젝트 매뉴얼

오픈소스기반기초설계 25-2학기

Digital Key에서 NFC를 이용한 Relay Attack

3조

소프트웨어학부 20223124 노다영

소프트웨어학부 20203096 김지윤

컴퓨터학부 20213132 유승종

컴퓨터학부 20211820 조강모

<목 차>

1. 프로젝트 제목, 팀 번호, 팀 구성원 소개
2. 기존 서비스 문제 제기
3. 구현된 프로젝트의 풀버전 GUI와 상세 메뉴얼
4. 기대 효과
5. Github URL

1. 프로젝트 제목, 팀 번호, 팀 구성원 소개

1-1. 프로젝트 제목

Digital Key에서 NFC를 이용한 Relay Attack

1-2. 팀 번호

3조

1-3. 팀 구성원 소개

노다영

- 소프트웨어학부 20223124
- 팀장
- 전반적인 문서 및 발표 자료 작성
- 실험 설계

김지윤

- 소프트웨어학부 20203096
- 실험 환경 구축
- 실험 수행

유승종

- 컴퓨터학부 20213132
- 자료 조사
- 실험 수행

조강모

- 컴퓨터학부 20211820
- 실험 환경 구축
- 실험 결과 정리

2. 기존 서비스 문제 제기

2-1. 사용자 인식 부족

현재 Digital Key 및 NFC 기반 인증 시스템은 높은 편의성을 바탕으로 빠르게 확산되고 있으나, 이에 비해 일반 사용자의 보안 인식 수준은 충분히 높지 않은 상황이다. 대부분의 사용자는 NFC가 근거리 통신이라는 점에 의존하여 “물리적으로 가까이 있어야만 인증이 가능하다”는 전제를 당연하게 받아들이며, NFC 통신이 중계될 수 있다는 가능성 자체를 인지하지 못하고 있다. 또한 사용자들은 보안 기능보다는 사용 편의성과 즉각적인 접근성을 우선시하는 경향이 강해, 보안 경고나 추가 인증 절차가 존재할 경우 이를 불편 요소로 인식하는 경우가 많다. 이러한 인식 구조는 NFC Relay Attack과 같은 공격이 현실적으로 가능함에도 불구하고, 사용자가 스스로 위험을 인지하거나 예방 행동을 취하기 어렵게 만드는 요인으로 작용한다.

2-2. 기업의 보안 투자 부족

기업 측면에서도 무선 통신 기반(NFC, UWB 등) Digital Key 시스템의 보안은 선제적 검증보다는 비용 효율성과 기능 구현 중심으로 설계되는 경향이 짙다. 대다수의 기업은 관련 표준이 요구하는 최소한의 보안 사양만을 충족하여 제품을 출시하며, 고도화된 공격 시나리오에 대비한 심층적인 보안 검증이나 추가적인 방어 로직 설계는 우선순위에서 배제하곤 한다. 이러한 접근은 초기 개발 비용을 절감할 수 있을지 모르나, 기술 자체의 이론적 보안성만을 맹신하여 실제 물리적 환경에서 발생하는 취약점을 간과하게 만든다. 이러한 현상의 근본적인 원인은 기업의 구조적인 투자 불균형과 출시 일정 압박에서 기인한다. Ponemon Institute와 SAE International이 공동으로 수행한 연구 보고서 'Securing the Modern Vehicle¹⁾'에 따르면, 실제 산업 현장의 데이터는 기업이 보안 내재화에 필요한 자원을 충분히 투입하지 않고 있음을 명확히 보여준다.

첫째, 경영진의 출시 속도 우선주의가 보안 검증을 무력화하고 있다. 해당 조사에 참여한 자동차 보안 전문가의 71%는 보안 취약점이 발생하는 가장 주된 원인으로 '촉박한 제품 출시 일정 압박'을 지목했다. 이는 기업이 UWB와 같은 신기술을 도입할 때, 충분한 시간을 들여 릴레이 공격(Relay Attack) 등의 물리적 취약점을 검증하기보다는, 시장 선점을 위해 검증 단계를 축소하거나 생략하는 경향이 만연해 있음을 방증한다.

둘째, 고도화된 기술 도입 속도에 비해 실질적인 보안 예산과 인력은 턱없이 부족하다. 화려한 편의 기능 구현에는 막대한 R&D 비용이 투입되지만, 정작 이를 방어할 보안 자원은 결핍되어 있다. 보고서에 따르면, 전체 응답자의 과반수인 51%가 조직 내에 사이버 보안 위협을 해결하기 위한 예산 및 인적 자원이 충분하지 않다고 응답했다. 특히 보안 인력 부족은 62%에 달해, 고도화되는 해킹 기법을 방어할 전문성이 기업 내부에서 확보되지 못하고 있음을 시사한다.

셋째, 보안 투자가 '선제적 예방'이 아닌 '사후 수습' 형태로 이루어지고 있다. 기업이 보안 예산을 증액하게 만드는 결정적인 요인을 묻는 설문에서, 자발적인 기술적 필요성보다는 '강제 리콜(60%)'이나 '심각한 해킹 사고 발생(54%)'과 같은 치명적인 손실이 발생한 이후에야 투자가 이루어진다는 응답이 지배적이었다. 이는 기업들이 잠재적인 위협을 알고 있음에도 불구하고, 실제 사고가 터지기 전까지는 보안을 비용 절감의 대상으로만

1) https://legacy.sae.org/binaries/content/assets/cm/content/topics/cybersecurity/securing_the_modern_vehicle.pdf

여기는 수동적인 경영 태도를 유지하고 있음을 보여주는 결정적인 지표다.

결론적으로, 현재의 Digital Key 시스템 취약점은 단순한 기술적 결함이 아니라, '출시 일정 준수'와 '비용 절감'을 최우선 가치로 두는 기업의 불균형한 투자 구조가 만들어낸 예견된 위험이라 할 수 있다.

2-3. 실무적 비용 및 영향

NFC Relay Attack이 실제 환경에서 성공할 경우, 차량 도난, 무단 출입과 같은 직접적이고 실질적인 피해가 즉시 발생할 수 있다. 특히 Digital Key는 차량 접근 및 시동 제어와 직접적으로 연결되어 있어, 단일 보안 사고가 사용자 개인의 재산 피해로 직결되는 위험성을 내포하고 있다. 더 나아가 이러한 사고는 보험료 상승, 브랜드 신뢰도 하락, 법적 분쟁 가능성 등 기업과 사용자 모두에게 2차적·장기적 손실로 이어질 수 있다. 즉, Relay Attack은 단순한 기술적 취약점을 넘어, 현실적인 경제적 비용과 사회적 영향을 수반하는 문제로 인식될 필요가 있다.

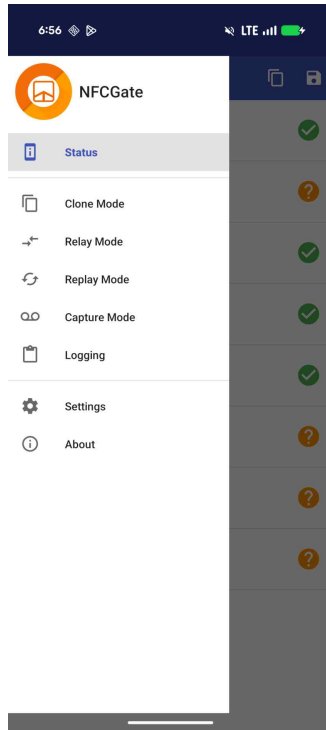
2-4. 사후 대응 중심의 보안 구조

현재 NFC 기반 Digital Key 보안 체계는 공격 발생 이후에야 문제를 인식하고 대응하는 사후 대응 중심 구조에 머무르는 경우가 많다. 그러나 보안 분야에서 공격 사례는 단순한 사고가 아니라, 취약점을 식별하고 방어 기술을 발전시키기 위한 중요한 연구 출발점이 된다. 특히 통제된 환경에서의 공격 실험과 검증은 실제 피해를 발생시키기 이전에 보안 취약성을 확인하고, 효과적인 예방적 보안 대책을 설계하기 위한 필수적인 과정으로 간주된다. 그럼에도 불구하고 이러한 실험적 접근이 충분히 이루어지지 않고 있다는 점은 현재 Digital Key 보안 구조의 한계를 명확히 보여준다.

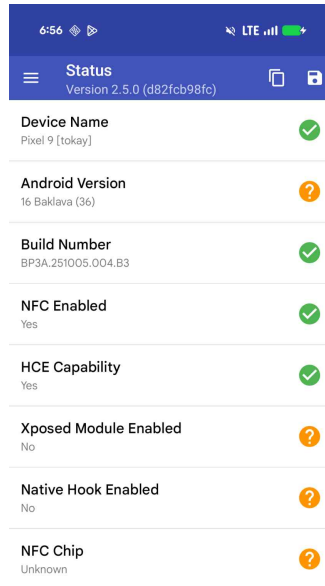
3. 구현된 프로젝트의 풀버전 GUI와 상세 매뉴얼

3-1. 구현된 프로젝트의 풀버전 GUI

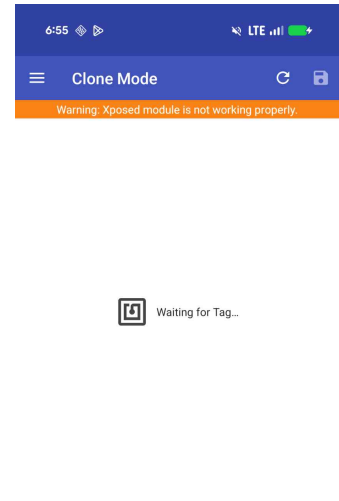
3-1-1. 어플리케이션 GUI



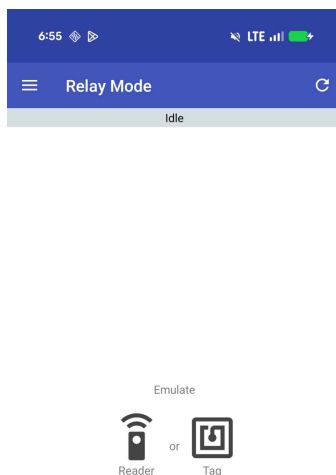
[그림 1] 메뉴바 화면



[그림 2] Status 화면



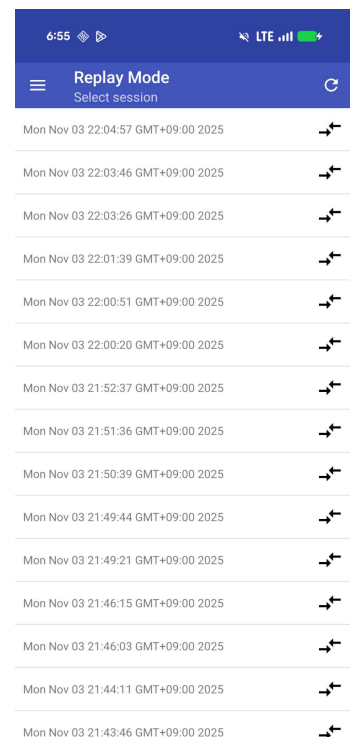
[그림 3] Clone Mode 화면



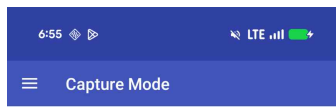
[그림 4] Relay Mode 화면



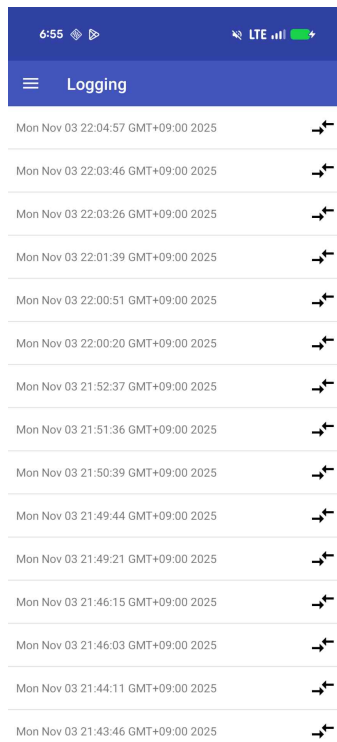
[그림 5] Relay Mode 실행 화면



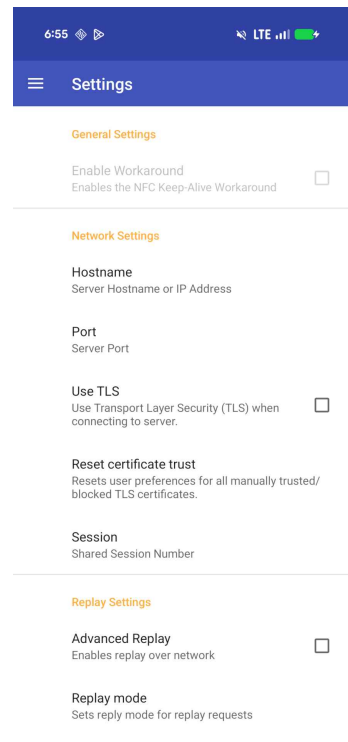
[그림 6] Replay Mode 화면



[그림 7] Capture Mode 화면



[그림 8] Logging 화면



[그림 9] Settings 화면

[그림 1]은 메뉴바 화면으로 Status 화면, Clone Mode 화면 등 여러 화면으로의 이동을 가능하게 한다.

[그림 2]는 Status 화면으로 현재 NFCGate가 설치된 핸드폰에서 어떤 권한이 허용되어 있고, 해당 핸드폰의 사양과 관련된 내용을 출력한다.

[그림 3]은 Clone Mode 화면으로 NFCGate가 설치된 핸드폰에 NFC Chip을 갖다 대면 해당 NFC의 정보를 clone 해오는 기능을 활성화 시키는 화면이다.

[그림 4]는 Relay Mode 화면으로 relay 공격을 할 때 사용되며 해당 핸드폰을 Reader를 emulate할지 tag를 emulate 할지 정한다.

[그림 5]는 어떤 것을 emulate할지 선택한 이후에 패킷이 오고 갈 때 화면이다.

[그림 6]은 Replay Mode 화면으로 replay 공격을 할 때 사용된다.

[그림 7]은 Capture Mode 화면으로 현재 NFCGate가 설치된 핸드폰에서 일어나는 NFC 로그를 capture 할 때 사용된다.

[그림 8]은 Logging 화면으로 NFCGate에서 일어난 로그들을 모아볼 수 있다.

[그림 9]는 Settings 화면으로 relay mode를 할 때 지정해야 하는 hostname을 설정하거나 port 등 여러 기능에서 필요한 설정을 할 때 사용된다.

3-1-2. 서버 GUI

timestamp	time_delta	command_type	CLA	INS	P1	P2	Lc	payload	protocol	vehicle	transacti	vehicle	endpoint	cryptogr	curve_poi	vehicle	command_n	response	status
2025-12-09 23:08:23.0	0.14s	UNKNOWN						330408c4...											
2025-12-09 23:08:27.4	0.16s	UNKNOWN						330408c4...											
2025-12-09 23:08:34.1	0.25s	UNKNOWN						330408c4...											
2025-12-09 23:08:39.2	0.24s	UNKNOWN						330408c4...											
2025-12-09 23:08:44.5	0.16s	UNKNOWN						330408c4...											

[그림 10] 서버측 실시간 화면

[그림 10]은 컴퓨터에서 가동중인 NFCGate 서버측 화면으로 timestamp, 받아서 전송한 패킷간의 시간차를 나타내는 time_delta, 해당 패킷이 어떤 command인지 나타내는 command_type 등 여러 정보를 실시간으로 출력하는 화면이다.

3-2. 구현된 프로젝트의 상세 매뉴얼

사용하는 핸드폰마다, 컴퓨터마다 각 절차는 약간의 차이가 있을 수 있다.

현재 아래 서술된 매뉴얼은 Google Pixel 9(핸드폰), Google Pixel 9 Pro XL(핸드폰), Window 10 기반의 데스크탑(3-2-1 절차에 사용), Linux 기반의 데스크탑(3-2-4 절차에 사용), Macbook Pro 16(3-2-4, 3-2-6, 3-2-7 절차에 사용)을 기준으로 한다.

3-2-1. 공격자 핸드폰 루팅

- (1) 핸드폰에서 ‘설정 → 휴대전화 정보 → 빌드번호 7회 탭하기 → 개발자 옵션 활성화’ 한다.
- (2) 컴퓨터에 ‘Android Debug Bridge(ADB)’를 다운로드한다.
- (3) <https://developers.google.com/android/images?hl=ko> 해당 URL에 들어가서 루팅하고자 하는 핸드폰에 대한 factory images를 다운로드 후 해당 factory images를 핸드폰으로 복사한다.
- (4) 핸드폰에서 <https://github.com/topjohnwu/Magisk/releases/tag/v30.6> 해당 URL에 접속해서 ‘Magisk-v30.6.apk’을 다운로드 한다.
- (5) 핸드폰에서 ‘Magisk-v30.6.apk’가 설치된 폴더로 가서 해당 파일을 눌러 핸드폰에 Magisk 앱을 설치한다.
- (6) 핸드폰에서 Magisk 앱에 접속해서 Magisk 필드에 있는 ‘설치’ 버튼을 누른다.
- (7) ‘파일 선택 및 패치’ 버튼을 누르고 (3)번 절차에서 핸드폰에 복사한 factory images를 선택한다. ‘magisk_patched-’로 시작하는 ‘.img’ 확장자 형식의 파일이 핸드폰에 설치된다.
- (8) 핸드폰과 (2)번 절차에서 ADB를 설치한 컴퓨터와 연결한다. 이때 핸드폰에 RSA 허용창이 뜰텐데 꼭 허용해줘야 한다.
- (9) 7번 절차 후 생성된 임의의 파일을 컴퓨터로 복사한다.
- (10) 컴퓨터에서 ‘명령 프롬프트’ 프로그램을 작동시키고 ‘adb reboot bootloader’ 명령어를 입력한다.
- (11) ‘명령 프롬프트’ 프로그램에서 ‘fastboot getvar current-slot’ 명령어를 입력해서

현재 슬롯이 무엇인지 확인한다.

- (12) (11)번 절차에서 확인한 현재 슬롯이 a라면 '명령 프롬프트' 프로그램에서 'fastboot flash init_boot_a magisk_patched.img' 명령어를 입력하고, b라면 'fastboot flash init_boot_b magisk_patched.img' 명령어를 입력한다.
- (13) '명령 프롬프트' 프로그램에서 'fastboot reboot' 명령어를 입력한다.
- (14) Google Play 스토어에서 Root Checker 앱을 다운 받아서 핸드폰 루팅이 완료되었는지 확인한다.
- (15) 해당 절차를 Google Pixel 9와 Google Pixel 9 Pro에서 각각 진행한다.

3-2-2. 공격자 핸드폰에 NFCGate 설치

- (1) 핸드폰에서 <https://github.com/NFC-Relay-Attack/nfcgate/releases/tag/nfc> 해당 URL에 들어가서 'NFCGate.2.5.0.apk'를 다운로드한다.
- (2) 핸드폰에서 다운로드된 NFCGate.2.5.0.apk를 실행한다.
- (3) 해당 절차를 Google Pixel 9와 Google Pixel 9 Pro에서 각각 진행한다.

3-2-3. 공격자 핸드폰에 LSPosed 설치

- (1) 핸드폰에서 https://github.com/mywalkb/LSPosed_mod/releases/tag/v1.9.3_mod 해당 URL에 들어가서 'LSPosed-v.1.9.3_mod-7244-zygisk-release.zip'을 다운로드한다.
- (2) 핸드폰에서 3-2-1 절차에서 설치한 Magisk 앱에 들어가서 '설정 → zygisk 활성화'를 한 후 재부팅을 한다.
- (3) 핸드폰에서 Magisk 앱에 들어가서 '모듈 → 저장소에서 설치' 버튼을 눌러 (1)번 절차에서 설치한 zip 파일을 선택하고 설치가 완료되면 재부팅을 한다.
- (4) 재부팅 후 핸드폰에서 알림창을 내리면 LSPosed Manager와 관련된 알림탭이 있을텐데 이를 선택하고 LSPosed Manager 앱을 설치한다.
- (5) LSPosed Manager 앱에서 3-2-2 절차에서 설치한 NFCGate를 선택한다.
- (6) 해당 절차를 Google Pixel 9와 Google Pixel 9 Pro에서 각각 진행한다.

3-2-4. 공격자 컴퓨터에 NFCGate server 설치

- (1) NFCGate server를 구동할 컴퓨터에서 <https://github.com/NFC-Relay-Attack/server> 해당 URL에 들어가서 Code → Download ZIP을 누른다.
- (2) Terminal을 켜고 1번 절차에서 다운로드 한 폴더로 이동해서 'python -m venv venv' 명령어를 통해서 파이썬 가상환경을 생성한다.
- (3) Terminal에서 'source venv/bin/activate' 명령어를 입력해서 가상환경을 활성화한다.
- (4) 필요한 패키지를 다운 받는다.

3-2-5. VPN 서버 구축

- (1) Docker를 설치한다.
- (2) VPN 설정을 저장할 폴더를 생성한다.
- (3) Terminal에서 해당 폴더로 이동하고 'vpn.env' 파일을 생성하는데, VPN_CLIENT

_NAME, 'docker run --name ipsec-vpn-server --restart=always -v ~/vpn-data:/etc/ipsec.d -p 500:500/udp -p 4500:4500/udp -d --privileged hwdsl2/ipsec-vpn-server' 명령어를 입력한다.

- (4) Terminal에서 'docker logs ipsec-vpn-server'를 입력해서 Server IP, IPsec PSK, Username, Password, Password for client config files 값을 확인한다.

3-2-6. 실험 환경 세팅

- (1) 공격용으로 NFCGate를 설치할 핸드폰 2개, 서버용으로 NFCGate server를 설치할 컴퓨터 1개, Victim 역할을 할 Digital Key를 갖고 있는 핸드폰 1개를 준비한다.
(편의상 reader 측을 emulate할 핸드폰 1대를 NG-R, tag 측을 emulate할 핸드폰 1대를 NG-T, NFCGate server를 설치한 컴퓨터를 NG-S라고 명명한다.)
- (2) 3-2-5 절차에서 설치한 VPN에서 생성된 .p12 파일을 NG-R, NG-T에 복사한다.
- (3) 핸드폰에서 복사해 온 .p12 파일을 누르고 3-2-5 절차에서 VPN 서버를 구축할 때 나왔던 비밀번호를 입력하고, 'VPN 및 앱 사용자 인증서'를 누르고 인증서 이름을 입력 되어있는대로 두고 '확인' 버튼을 누른다.
- (4) 핸드폰에서 '설정 → 네트워크 및 인터넷 → VPN'으로 들어가서 오른쪽 상단에 + 버튼을 누른다.
- (5) 이름은 digitalkey로 설정하고, 유형은 'IKEv2/IPSec RSA'를 선택하고, 서버주소는 3-2-5 절차에서 VPN 서버를 구축할 때 나왔던 Server IP를 입력하고, IPSec 식별자에 digitalkey를 입력하고, IPSec CA 인증서와 IPSec 서버 인증서를 (3)번 절차에서 설치된 인증서를 선택한다.
- (6) (2)번부터 (5)번까지의 절차를 다른 핸드폰에서도 반복한다.
- (7) 3-2-5 절차에서 설치한 VPN에서 생성된 .mobileconfig 파일을 NG-S 컴퓨터에 복사한다.
- (8) (3)번부터 (5)번까지의 절차를 NG-S 컴퓨터에서 반복한다.

3-2-7. 실험 수행

- (1) NG-R, NG-T, NG-S 3개의 기기를 모두 VPN에 연결한다.
- (2) NG-S에서 Terminal에 접속해서 'python -u server.py log | tee '저장 파일 이름' | python parsing_NFC_delay.py --stdin' 명령어를 입력해서 NFCGate Server를 작동시킨다.
- (3) NG-S에서 Terminal에서 ifconfig를 쳐서 VPN이 연결된 현재 상태에서의 IP를 확인한다.
- (4) NG-R과 NG-T에서 NFCGate 앱을 실행하고 Settings 화면에서 Hostname 부분에 (3)번 절차에서 확인한 IP를 입력한다.
- (5) NG-R에서 NFCGate 앱을 실행하고 relay mode에 들어가서 emulate 대상으로 Reader를 선택한다.
- (6) NG-T에서 NFCGate 앱을 실행하고 relay mode에 들어가서 emulate 대상으로 Tag를 선택한다.
- (7) NG-R은 victim 핸드폰에 접근하고, NG-T는 차량에 접근한다.

4. 기대 효과

4-1. 학술적 기여

본 프로젝트는 NFCGate를 활용하여 차량 디지털 키 환경에서 발생 가능한 Relay Attack을 실제로 구현·분석함으로써, 이론적 위협에 머물러 있던 공격 시나리오를 실증적으로 검증하고 그 위험성을 명확히 규명하는 데 의의가 있다. 특히 차량용 NFC 기반 디지털 키는 표준화와 상용화가 빠르게 진행되고 있음에도 불구하고, 실환경에서의 공격 가능성과 취약점에 대한 체계적인 실증 연구는 상대적으로 부족한 상황이다. 본 연구는 NFC Relay 공격의 구체적인 동작 구조, 공격 성공 조건, 통신 지연 특성 등을 분석함으로써, 향후 디지털 키 보안 표준 수립과 차량 보안 연구의 기초 자료로 활용될 수 있는 실증적 근거를 제공한다. 더 나아가 학계와 산업계 간 공동 연구를 촉진하여, 차량 보안 분야 전반의 연구 수준을 한 단계 끌어올리는 데 기여할 것으로 기대된다.

4-2. 사용자 경각심 고취

또한 본 프로젝트는 디지털 키 사용 환경에 놓인 일반 사용자 및 차량 소유자의 보안 경각심을 제고하는 데 중요한 역할을 한다. 스마트폰 기반 디지털 키는 편의성을 중심으로 빠르게 확산되고 있으나, 사용자들은 해당 기술이 내포한 보안 위협에 대해 충분히 인지하지 못하는 경우가 많다. Relay Attack 실증 결과를 통해 디지털 키가 단순한 전자 편의 기능이 아니라, 물리적 차량 접근과 직결된 고위험 자산임을 명확히 인식하게 함으로써, 안전한 사용 습관 형성과 정기적인 소프트웨어 업데이트의 필요성을 자연스럽게 유도할 수 있다. 이는 단기적인 기술 대응을 넘어, 일상생활 전반에서의 보안 인식 수준을 향상시키는 사회적 효과로 이어질 수 있다.

4-3. 산업 투자 증진

산업적 측면에서도 본 연구는 차량 제조사 및 관련 부품·플랫폼 기업의 보안 투자 확대를 촉진하는 계기가 될 수 있다. Relay Attack의 실현 가능성과 파급 효과가 실증적으로 제시될 경우, 제조사들은 기존 기능 중심의 개발 전략에서 벗어나 보안을 핵심 경쟁 요소로 인식하게 될 가능성이 높다. 이를 통해 디지털 키 시스템에 대한 지속적인 패치 및 업데이트 체계가 강화되고, 보안 기술을 중심으로 한 차량 산업 전반의 경쟁력이 제고될 수 있다. 결과적으로 본 프로젝트는 차량 보안 기술이 비용 요소가 아닌, 브랜드 신뢰도와 시장 경쟁력을 좌우하는 핵심 요소임을 강조하는 실증 사례로 활용될 수 있다.

4-4. 방어 메커니즘 개발

마지막으로, 본 연구는 실질적인 방어 메커니즘 개발을 위한 기술적 시사점을 제공한다. Relay Attack 과정에서 발생하는 통신 지연 특성, 거리 기반 이상 징후, 시간 정보의 변조 가능성 등을 분석함으로써, Relay 탐지 및 지연 분석 기술의 필요성을 구체적으로 제안할 수 있다. 나아가 거리 증명(distance bounding) 기법이나 타임스탬프 인증과 같은 보안 메커니즘을 디지털 키 시스템에 적용하는 방안을 제시함으로써, 이론에 그치지 않는 실용적 보안 가이드라인 수립에 기여한다. 이는 향후 차량 디지털 키 보안 설계 시 참고 가능한 실질적 기준으로 작용하며, 궁극적으로는 Relay Attack에 강인한 차세대 차량 접근 제어 시스템 개발로 이어질 수 있을 것으로 기대된다.

5. Github URL

<https://github.com/orgs/NFC-Relay-Attack/repositories>